SPECIFICATION

Calypso Terminal API

Card

Version 3.0.0-SNAPSHOT, 2026-10-28

⚠ ***Warning. The present document cancels and replaces the previous release.***

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Link and Contact



Website
https://calypsonet.org



Support
support@calypsonet.org

Copyright © Calypso Networks Association, https://calypsonet.org.

This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** *(CC BY-ND 4.0).*

You are free to **share** *— copy and redistribute this document in any medium or format for any purpose, even commercially — under the following terms:*

- ***Attribution*** *— You MUST give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You MAY do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.*
- ***NoDerivatives*** *— If you remix, transform, or build upon this document, you MAY NOT distribute the modified material.*

No additional restrictions — You MAY NOT apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Implementation grant *— The* **NoDerivatives** *condition above applies to this specification* **document** *only — its text, tables and diagrams. It does* **not** *restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.*

The full license text is available at https://creativecommons.org/licenses/by-nd/4.0/.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Table of Contents

1. Abstract . . . 5
2. Introduction . . . 6
2.1. Purpose . . . 6
2.2. Scope . . . 6
2.3. Conformance . . . 6
2.4. Document Conventions . . . 7
2.4.1. Naming . . . 7
2.4.2. Data types . . . 7
2.4.3. Type tables . . . 8
2.4.4. Operation tables . . . 8
2.4.5. Operation contracts . . . 8
2.4.6. Concurrency . . . 9
2.5. References and Resources . . . 9
2.5.1. Calypso References . . . 9
2.5.2. Normative References . . . 9
2.5.3. External Resources . . . 9
3. Glossary and Acronyms . . . 10
3.1. Glossary . . . 10
3.2. Acronyms . . . 10
4. Architectural Overview . . . 11
4.1. Functional positioning . . . 11
4.2. Logical namespaces . . . 11
4.3. Multi-channel design . . . 11
4.4. APDU exchange execution-time control . . . 12
4.5. APDU construction rules . . . 12
4.6. Limitations and constraints . . . 12
4.7. UML class diagram . . . 12
5. API Specification . . . 14
5.1. Constants . . . 14
5.1.1. Card API Properties . . . 14
5.2. Data . . . 14
5.2.1. APDU Request . . . 14
5.2.2. APDU Response . . . 15
5.2.3. Card Request . . . 15
5.2.4. Card Response . . . 15
5.2.5. Card Selection Request . . . 16
5.2.6. Card Selection Response . . . 16
5.3. API Interfaces . . . 17
5.3.1. Proxy Reader API . . . 17
Close Channel . . . 18
Transmit Card Request . . . 18
Transmit Card Request And Close Channel . . . 18
5.4. SPI Interfaces . . . 19
5.4.1. Basic Card Selection Extension SPI . . . 19
Parse . . . 19
5.4.2. Card Selection Extension SPI . . . 19
Get Card Selection Request . . . 20

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.4.3. ISO Card Selection Extension SPI . . . 20
Parse . . . 20
5.4.4. ISO Smart Card SPI . . . 20
5.4.5. Multichannel Smart Card SPI . . . 20
Get Channel . . . 21
5.4.6. Smart Card SPI . . . 21
Deactivate . . . 21
5.5. Errors . . . 21
5.5.1. APDU Exchange Duration Exceeded . . . 22
5.5.2. Card Broken Communication . . . 22
5.5.3. Parse . . . 22
5.5.4. Reader Broken Communication . . . 22
5.5.5. Unexpected Status Word . . . 22

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

1. Abstract

This document is the official technical specification of the **Terminal Card API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Card API MUST satisfy.

The Terminal Card API is the **underside** of the **Terminal Reader API** (CNA-TR-API). It defines the **internal** contract used by reader implementations and by card extension modules to exchange APDU-level data with cards, while the **Terminal Reader API** remains the public-facing surface used by terminal applications.

Document Status

Reference	261028-SP-CNATerminalAPI-Card
Short name	CNA-TC-API
Version	3.0.0-SNAPSHOT
Revision date	2026-10-28
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-card-api-spec
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)

The present document specifies the 3.0.0-SNAPSHOT of the Terminal Card API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List

This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v3.0.0-SNAPSHOT 2026-10-28	Initial edition of this document.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2. Introduction

2.1. Purpose

The Terminal Card API defines the **internal** contract by which:

- a **reader implementation** MUST expose an APDU transmission interface (the **proxy reader**) so that selection managers and card transaction managers can exchange data with cards in a uniform way,
- a **card extension** MUST expose its selection process (request preparation and response parsing) so that the selection manager defined by CNA-TR-API can drive it,
- APDU-level errors carry the responses already received before the failure together with any duration information required to enforce the APDU exchange execution-time bounds.

The API is **internal**: terminal applications MUST NOT use it directly. It is the contract that allows reader implementations and card extensions written by independent vendors to interoperate.

2.2. Scope

This specification covers:

- the APDU request and response data carriers, including the per-request duration bound and per-response measured duration used to control the APDU exchange execution time,
- the selection request and response data carriers, including the multi-channel-aware channel information,
- the proxy-reader contract through which APDUs are transmitted, including the multi-channel transmission and closure operations,
- the SPI implemented by card extensions to drive selection,
- the SPI implemented by card extensions to represent a selected smart card, including the multi-channel variant,
- the family of errors raised at APDU exchange level.

The following topics are **out of scope**:

- the public reader interface exposed to terminal applications (covered by CNA-TR-API),
- the parsing of APDU payloads (deferred to specific card extension specifications),
- the protocol-level cryptographic security mechanisms handled by card extensions such as the Calypso Card API.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it raises errors exclusively of the types defined in Section 5.5 for the situations described,
3. it preserves the structural relationships described in Chapter 4,
4. every object it provides implements the interface of CNA-TR-API that corresponds to its type, as described below.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

The Terminal Reader API (CNA-TR-API) exposes to terminal applications the public-facing facet of the objects whose internal facet is defined by this specification. An implementation exposes each contract of this API through an adapter that also implements the corresponding interface of CNA-TR-API: every object implementing a type of the right column MUST therefore also implement the interface of the left column.

CNA-TR-API interface	CNA-TC-API type
`CardReader`	`ProxyReaderApi`
`BasicCardSelectionExtension`	`BasicCardSelectionExtensionSpi`
`IsoCardSelectionExtension`	`IsoCardSelectionExtensionSpi`
`SmartCard`	`SmartCardSpi`
`IsoSmartCard`	`IsoSmartCardSpi`

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order to avoid ambiguity.

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.7).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
`String`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`Boolean`	Two-state truth value.	`boolean`, `bool`.
`Int`	Signed 32-bit integer.	`int`, `Int32`, `i32`.
`Long`	Signed 64-bit integer.	`long`, `Int64`, `i64`.
`ByteArray`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`List<T>`	Ordered collection of `T` values, may contain duplicates.	`List<T>`, `Vec<T>`.
`Set<T>`	Unordered collection of unique `T` values.	`Set<T>`, `HashSet<T>`.
`Any`	Value of any type, root of the type hierarchy.	`Object`, `Any`, `object`, `Box<dyn Any>`.
`T?`	Nullable value: either a `T` value or `null`.	`Integer` (boxed), `Optional<T>`, `T?`, `Option<T>`.
`null`	Absence of value, the alternative held by a nullable type.	`null`, `nil`, `None`.
`Unit`	Indicates that an operation returns no value.	`void`, `Unit`, `()`.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).
Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller MUST guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
  - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
  - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
  - **State**—an illegal state error.
  - **Capability**—an unsupported operation error.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

- **Collections.** Unless the return type is marked nullable (`T?`), an operation whose return type is a collection (e.g. `List`, `Set`, `Map`) returns an empty collection rather than `null`.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

Reference	Document
[CNA-TR-API]	*CNA Terminal API — Reader* (SP-CNATerminalAPI-Reader), version 3.0, Calypso Networks Association.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816]	*Identification cards — Integrated circuit cards*.
[ISO/IEC 14443]	*Identification cards — Contactless integrated circuit cards — Proximity cards*.
[PC/SC]	*Interoperability Specification for ICCs and Personal Computer Systems*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
APDU	Application Protocol Data Unit, the message format used between the terminal and the card (ISO/IEC 7816-4).
Case 4 APDU	APDU command that carries data in both directions; the `Le` field is mandatory and RECOMMENDED to be set to `00h`.
Card Extension	Software module providing the card-specific dialog with a card family (e.g. Calypso).
Logical Channel	Software-level communication path established with a card after a successful selection.
Basic Channel	The default logical channel (channel `0`) of an ISO/IEC 7816-4 card.
Physical Channel	Hardware-level communication path between the reader and the card.
Proxy Reader	Reader-side facet exposing APDU transmission to extension layers, by opposition to the public reader interface presented to terminal applications.
Status Word (SW)	Two-byte trailer of an APDU response indicating the outcome of the command.
SPI	Service Provider Interface — a set of interfaces designed to be implemented by an extension module.

3.2. Acronyms

Abbreviation	Expansion
APDU	Application Protocol Data Unit
ATR	Answer To Reset
CNA	Calypso Networks Association
FCI	File Control Information
ISO	International Organization for Standardization
SPI	Service Provider Interface
SW	Status Word
UML	Unified Modelling Language

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4. Architectural Overview

4.1. Functional positioning

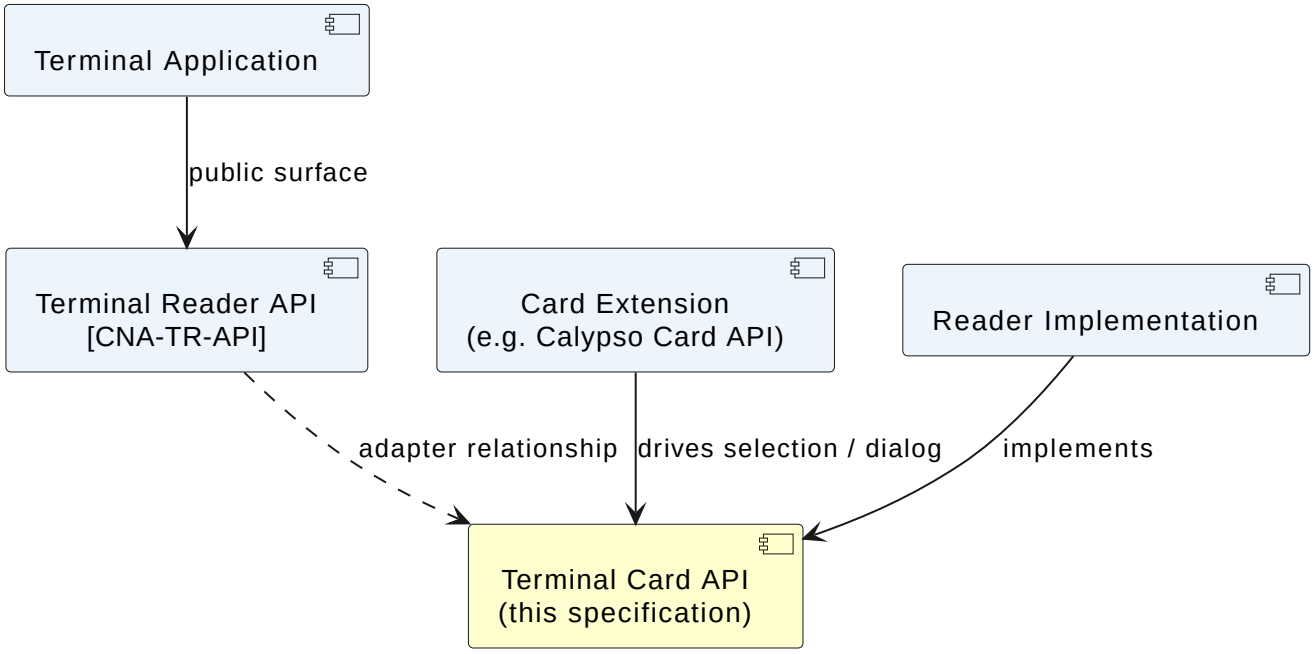


Figure 1. Terminal Card API — Architecture Overview

The **adapter relationship** depicted above (`TR ..> TC`) binds this specification to CNA-TR-API: an implementation exposes each contract of this API through an adapter that also implements the corresponding public-facing interface of the **Terminal Reader API**. These correspondences are listed in the **Conformance** section.

4.2. Logical namespaces

Namespace	Role	Summary
`card`	Public API (internal-only audience)	Properties, parse error, data carriers (APDU response, card response, selection response), proxy-reader interface and errors.
`card.spi`	SPI	Data carriers built by card extensions, and the contracts they implement.

4.3. Multi-channel design

The Terminal Card API exposes the multi-channel capability of ISO/IEC 7816-4 cards through:

- a dedicated SPI sub-type `MultichannelSmartCardSpi` that carries the channel number of each active smart card,
- dedicated proxy-reader operations `transmitCardRequestAndCloseChannel` and `closeChannel` that target a specific channel by passing the corresponding `MultichannelSmartCardSpi`.

The single-channel transmission operation `transmitCardRequest` accepts the base `SmartCardSpi` and serves both the single-channel and the multi-channel use case (the implementation extracts the channel information from the dynamic sub-type when needed).

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4.4. APDU exchange execution-time control

This specification allows the execution time of each APDU exchange to be controlled at software level. It provides:

- an **optional** maximum duration on each request (`ApduRequest.apduExchangeMaxDuration`),
- an **optional** effective duration on each response (`ApduResponse.apduExchangeDuration`),
- a dedicated error, `ApduExchangeDurationExceeded`, raised when the effective duration of an exchange exceeds the declared maximum.

The durations exchanged through this specification are expressed in **microseconds**. The effective resolution of the measurement depends on the reader and on the execution platform: an implementation MUST document the resolution it provides, and a bound finer than that resolution MUST NOT be relied upon.

This mechanism is a practical way to implement security countermeasures against two threats. Against a **relay attack**, relaying the dialog with a card to a remote location adds a transmission delay; an abnormally long exchange can therefore reveal that the card is not actually present in front of the reader. Against **card emulation**, a generic RFID device answering in place of the expected card processes the command in software, where the chip answers in hardware; an abnormally long exchange then reveals that the answer does not come from the expected product. The maximum duration to declare depends on the command, the card and the reader, and is chosen by the application or the card extension.

4.5. APDU construction rules

The APDU commands carried by `ApduRequest` MUST strictly comply with ISO/IEC 7816-3. In particular, a case 4 command—one carrying data in both directions—MUST include the `Le` field. It is RECOMMENDED to set this field to `00h`.

4.6. Limitations and constraints

The Card API does not allow the transmission of the following APDU commands to a card to be requested:

- **Select Application** by DF name: the selection of a card application is managed exclusively through a `CardSelectionRequest`;
- **Get Response**: the status words `61XYh` and `6CXYh` are handled automatically by the reader implementation, which issues a **Get Response** command or re-issues the previous command with the expected `Le` field.

4.7. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

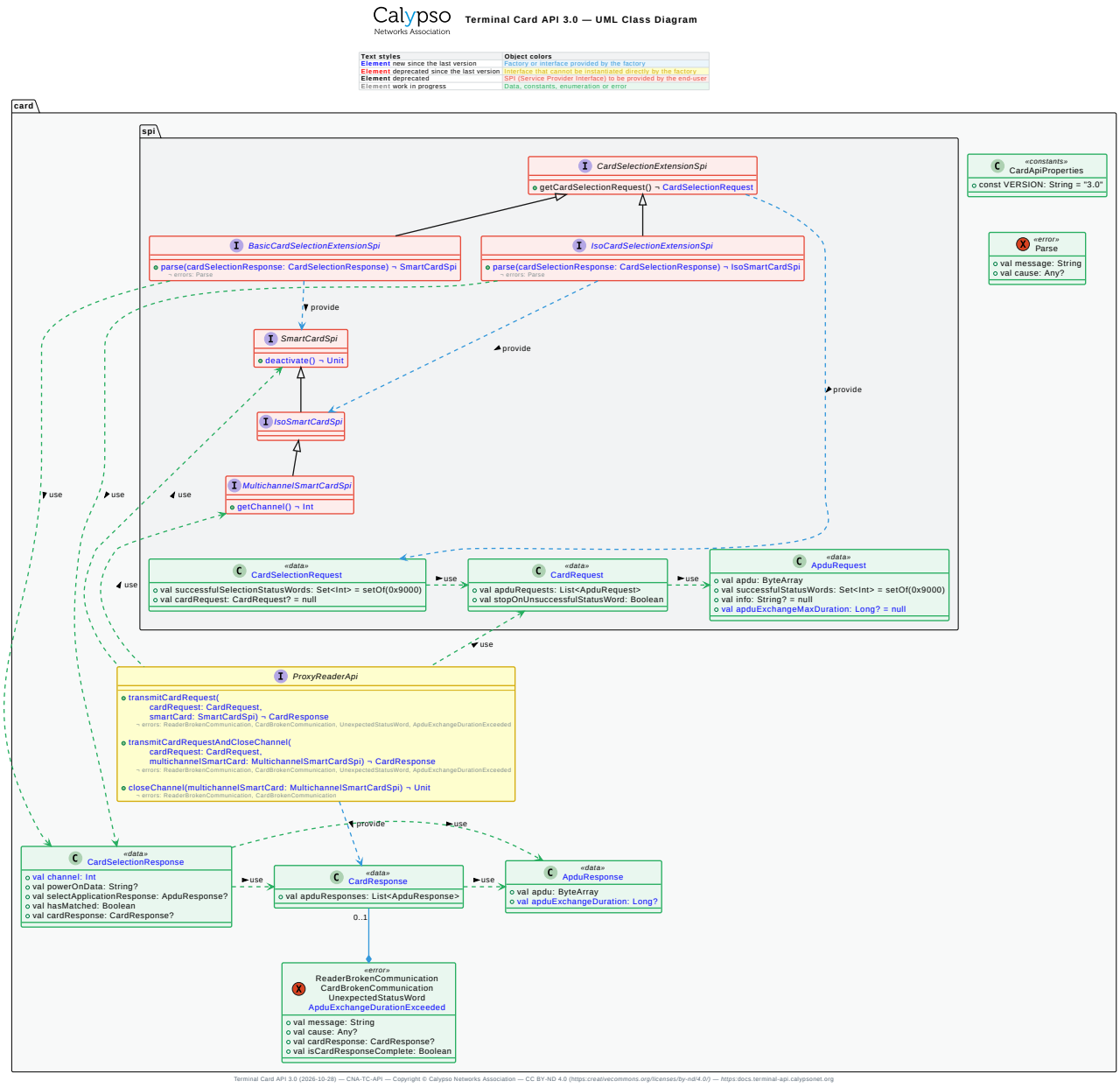


Figure 2. Terminal Card API — UML Class Diagram

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5. API Specification

5.1. Constants

5.1.1. Card API Properties

Name	`CardApiProperties`
Kind	Constants
Namespace	`card`
Since	1.0
Purpose	Exposes the immutable properties of the Terminal Card API.

Name	Type	Since	Description
`VERSION`	`String`	1.0	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form `MAJOR.MINOR`; it is `"3.0"` for this revision of the specification.

5.2. Data

5.2.1. APDU Request

Name	`ApduRequest`
Kind	Data
Namespace	`card.spi`
Since	1.0
Purpose	Carries a single APDU request together with its successful status words, its information string and an optional duration bound used to control the APDU exchange execution time.
See also	`ApduResponse`

Name	Type	Default	Since	Description
`apdu`	`ByteArray`	—	1.0	Bytes of the APDU command (header and, when applicable, data field and `Le`), built according to the APDU construction rules. At least 4 bytes.
`successfulStatusWords`	`Set<Int>`	`setOf(0x9000)`	1.0	Status words that MUST be considered successful for this APDU. Non-empty, containing at least `9000h`.
`info`	`String?`	`null`	1.0	Informational string describing the APDU, used by implementations for logging and error messages. Non-empty when set; `null` when no information is defined.
`apduExchangeMaxDuration`	`Long?`	`null`	3.0	Maximum tolerated duration of the APDU exchange, in **microseconds**. The proxy reader MUST measure the effective duration of the exchange and raise `ApduExchangeDurationExceeded` if it exceeds this bound. Strictly positive when set; `null` when no bound is defined for this request.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5.2.2. APDU Response

Name	`ApduResponse`
Kind	Data
Namespace	`card`
Since	1.0
Purpose	Carries the data received in response to a single APDU command (variable-length payload followed by the status word).
See also	`ApduRequest`

Instances are serialisable so that they can be transported across distributed boundaries.

Name	Type	Default	Since	Description
`apdu`	`ByteArray`	—	1.0	Raw response bytes received from the card, **including** the status word: the last two bytes hold the status word, in the range `[0x0000, 0xFFFF]`, and the preceding bytes hold the data part, empty when the response carries no data. At least 2 bytes.
`apduExchangeDuration`	`Long?`	—	3.0	Effective duration of the APDU exchange, in **microseconds**, as measured by the underlying reader implementation. `null` if the reader does not provide the measurement.

5.2.3. Card Request

Name	`CardRequest`
Kind	Data
Namespace	`card.spi`
Since	1.0
Purpose	Aggregates an ordered list of `ApduRequest` to be executed consecutively against the card, together with a flag indicating whether processing stops when one of the APDUs answers with an unexpected status word.
See also	`CardResponse`

Name	Type	Default	Since	Description
`apduRequests`	`List<ApduRequest>`	—	1.0	Ordered list of the APDU requests to be executed. Non-empty.
`stopOnUnsuccessfulStatusWord`	`Boolean`	—	1.0	Indicates whether the iteration MUST stop at the first APDU whose response status word is not in its `successfulStatusWords` set: `true` to stop on the first failure, `false` to continue regardless.

5.2.4. Card Response

Name	`CardResponse`
Kind	Data
Namespace	`card`
Since	1.0
Purpose	Groups multiple APDU responses received from the card after the execution of a `CardRequest`.
See also	`CardRequest`

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Instances are serialisable so that they can be transported across distributed boundaries.

A `CardResponse` MAY be carried by an APDU-level error; in that case some responses MAY be missing (e.g. the card was removed during processing, or processing stopped after an unsuccessful status word—see `CardRequest.stopOnUnsuccessfulStatusWord`).

Name	Type	Default	Since	Description
`apduResponses`	`List<ApduResponse>`	—	1.0	Responses received for the executed APDU requests, in the same order in which the requests were submitted. Empty if there is no response.

5.2.5. Card Selection Request

Name	`CardSelectionRequest`
Kind	Data
Namespace	`card.spi`
Since	1.0
Purpose	Carries the data needed to execute a single selection case: the status words considered successful for the selection, and an optional card request to be executed after a successful card selection.
See also	`CardSelectionResponse`

Name	Type	Default	Since	Description
`successfulSelectionStatusWords`	`Set<Int>`	`setOf(0x9000)`	2.0	Status words that MUST be considered successful for this selection case. Non-empty, containing at least `9000h`.
`cardRequest`	`CardRequest`?	`null`	1.0	Optional follow-up card request to execute after a successful selection. `null` if no follow-up is needed.

5.2.6. Card Selection Response

Name	`CardSelectionResponse`
Kind	Data
Namespace	`card`
Since	1.0
Purpose	Carries the data observed during the start-up phase with the card—the selection step itself, the channel on which the card has been placed and any additional commands that may have been executed afterwards.
See also	`CardSelectionRequest`

Name	Type	Default	Since	Description
`channel`	`Int`	—	3.0	Logical channel number on which the card has been placed by the selection: `0` for a single-channel selection scenario; for a multi-channel selection scenario, any value supported by the underlying ISO/IEC 7816-4 card. Non-negative.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Name	Type	Default	Since	Description
`powerOnData`	`String?`	—	1.0	Card's power-on data, that is, the data retrieved by the reader when the card is inserted. For a contact reader, this is the Answer To Reset (ATR) defined by ISO/IEC 7816. For a contactless reader, the reader decides what this data is. Some contactless readers provide a virtual ATR (partially standardised by the PC/SC standard), but other devices MAY have their own definition, including for example elements from the anti-collision stage of the ISO/IEC 14443 protocol (ATQA, ATQB, ATS, SAK, etc.) or any proprietary definition. As this data varies from one reader to another, it is exposed as a `String`, which MAY be a hexadecimal string or any other relevant representation. Non-empty when set; `null` if no power-on data is available.
`selectApplicationResponse`	`ApduResponse?`	—	1.0	Response received from the card to the **Select Application** command. `null` if no **Select Application** command was executed.
`hasMatched`	`Boolean`	—	1.0	Indicates whether the inserted card matches the selection filters.
`cardResponse`	`CardResponse?`	—	1.0	Responses received for any additional commands attached to the selection case. `null` if no additional command was executed.

5.3. API Interfaces

5.3.1. Proxy Reader API

Name	`ProxyReaderApi`
Kind	Interface
Namespace	`card`
Since	1.0
Purpose	Reader-side facet exposing APDU transmission and channel-closure operations to card extensions.

A `ProxyReaderApi` and the `CardReader` interface defined in CNA-TR-API are two facets of the same underlying reader; they are separated so that only the **internal** dialog with cards is exposed to extension modules. `ProxyReaderApi` is the **backside** of `CardReader`: it is the reader able to transmit card requests and having control over the physical channel.

The proxy reader uses the **smart card SPI** received as parameter to identify the target of every operation: it extracts the channel information (when the dynamic type is `MultichannelSmartCardSpi`) and verifies the active state of the smart card (see the role of the smart-card SPI parameter) before performing any exchange.

Role of the smart-card SPI parameter

The three operations of `ProxyReaderApi` receive a smart-card SPI parameter. This parameter is **not** a mere vehicle for the channel number. It plays up to three roles:

1. **Channel routing** (only when the dynamic type is `MultichannelSmartCardSpi`): the logical channel number is read via `getChannel()` and used to route the APDU exchange or the closure on the correct logical channel. When the parameter is a base `SmartCardSpi` (not a `MultichannelSmartCardSpi`), the exchange is addressed to the basic channel (channel `0`).
2. **Active-state check**: the active state of the smart card is verified before any exchange (`SmartCard.isActive()` is `true` per CNA-TR-API). If it is not active, a `CardBrokenCommunication` is raised without contacting the card.
3. **Deactivation**: when the situation requires it (typically after a communication failure or after an explicit channel closure), `SmartCardSpi.deactivate` is called so that the deactivation is immediately visible to the

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

application via `SmartCard.isActive() == false`.

Roles 2 and 3 are common to `SmartCardSpi` and `MultichannelSmartCardSpi`; role 1 is specific to the multi-channel variant.

Close Channel

Signature	`closeChannel(multichannelSmartCard: MultichannelSmartCardSpi) → Unit`
Since	3.0
Description	Closes the logical channel of the smart card identified by `multichannelSmartCard` and deactivates the smart card. This operation is idempotent: calling it on an already-closed channel has no effect.
Parameters	`multichannelSmartCard` (`MultichannelSmartCardSpi`) — the multi-channel smart card SPI identifying the target.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`ReaderBrokenCommunication` — if communication with the reader is broken. `CardBrokenCommunication` — if communication with the card is broken.

Transmit Card Request

Signature	`transmitCardRequest(`
	`  cardRequest: CardRequest,`
	`  smartCard: SmartCardSpi`
	`) → CardResponse`

Since	3.0
Description	Transmits the provided `CardRequest` targeting the smart card identified by `smartCard`. The logical channel remains open after the operation. If the smart card is no longer active at the moment of the call, a `CardBrokenCommunication` is raised without any exchange. **Note:** if an error occurs while sending an APDU (communication error or unexpected status word), an APDU-level error is raised and the responses already collected from the previously transmitted APDUs are attached to it. This lets the application tolerate card tearing and retrieve the partial response, or keep strict control over the APDUs sent to the card (see `CardRequest.stopOnUnsuccessfulStatusWord`).
Parameters	`cardRequest` (`CardRequest`) — the request to transmit. `smartCard` (`SmartCardSpi`) — the smart card SPI identifying the target.
Returns	A `CardResponse`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`ReaderBrokenCommunication` — if communication with the reader is broken. `CardBrokenCommunication` — if communication with the card is broken, or if the smart card is no longer active. `UnexpectedStatusWord` — if a command returned an unexpected status word. `ApduExchangeDurationExceeded` — if a measured APDU duration exceeded the bound declared on the request.

Transmit Card Request And Close Channel

Signature	`transmitCardRequestAndCloseChannel(`
	`  cardRequest: CardRequest,`
	`  multichannelSmartCard: MultichannelSmartCardSpi`
	`) → CardResponse`

Since	3.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Transmits the provided request targeting the smart card identified by `multichannelSmartCard` and, upon successful completion, closes the corresponding logical channel. The smart card is deactivated after the channel has been closed (see the role of the smart-card SPI parameter).
Parameters	`cardRequest` (`CardRequest`) — the request to transmit. `multichannelSmartCard` (`MultichannelSmartCardSpi`) — the multi-channel smart card SPI identifying the target.
Returns	A `CardResponse`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`ReaderBrokenCommunication` — if communication with the reader is broken. `CardBrokenCommunication` — if communication with the card is broken, or if the smart card is no longer active. `UnexpectedStatusWord` — if a command returned an unexpected status word. `ApduExchangeDurationExceeded` — if a measured APDU duration exceeded the bound declared on the request.

5.4. SPI Interfaces

5.4.1. Basic Card Selection Extension SPI

Name	`BasicCardSelectionExtensionSpi`
Kind	Interface (SPI)
Namespace	`card.spi`
Extends	`CardSelectionExtensionSpi`
Since	3.0
Purpose	Selection SPI of a card extension whose products are selected without ISO/IEC 7816-4 filters. It is the counterpart of the `BasicCardSelectionExtension` of CNA-TR-API.

Parse

Signature	`parse(cardSelectionResponse: CardSelectionResponse) → SmartCardSpi`
Since	3.0
Description	Parses the response of an executed selection case to produce the corresponding smart-card SPI.
Parameters	`cardSelectionResponse` (`CardSelectionResponse`) — the card selection response.
Returns	A `SmartCardSpi`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`Parse` — if the response cannot be interpreted.

5.4.2. Card Selection Extension SPI

Name	`CardSelectionExtensionSpi`
Kind	Interface (SPI)
Namespace	`card.spi`
Since	2.0
Purpose	Drives a selection case from the card-extension side: builds the selection request. The parsing of the response is defined by the two family SPIs that extend it, `BasicCardSelectionExtensionSpi` and `IsoCardSelectionExtensionSpi`, each producing the smart-card SPI of its family.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Get Card Selection Request

Signature	`getCardSelectionRequest() → CardSelectionRequest`
Since	1.0
Description	Returns the request to use when executing this selection case.
Returns	A `CardSelectionRequest`.
Pre-conditions	None.
Errors	None.

5.4.3. ISO Card Selection Extension SPI

Name	`IsoCardSelectionExtensionSpi`
Kind	Interface (SPI)
Namespace	`card.spi`
Extends	`CardSelectionExtensionSpi`
Since	3.0
Purpose	Selection SPI of a card extension whose products are ISO/IEC 7816-4 cards. It is the counterpart of the `IsoCardSelectionExtension` of CNA-TR-API, and the only family able to produce a smart card active on a logical channel.

Parse

Signature	`parse(cardSelectionResponse: CardSelectionResponse) → IsoSmartCardSpi`
Since	3.0
Description	Parses the response of an executed selection case to produce the corresponding smart-card SPI. The result is an `IsoSmartCardSpi`, which the extension MAY specialise further as a `MultichannelSmartCardSpi` when the card supports several active logical channels.
Parameters	`cardSelectionResponse` (`CardSelectionResponse`) — the card selection response.
Returns	An `IsoSmartCardSpi`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	`Parse` — if the response cannot be interpreted.

5.4.4. ISO Smart Card SPI

Name	`IsoSmartCardSpi`
Kind	Marker interface (SPI)
Namespace	`card.spi`
Extends	`SmartCardSpi`
Since	3.0
Purpose	Card-extension facet of a smart card selected through ISO/IEC 7816-4 filters. It declares no operation of its own; it is the type from which the multi-channel facet derives.

5.4.5. Multichannel Smart Card SPI

Name	`MultichannelSmartCardSpi`
Kind	Interface (SPI)

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Namespace	`card.spi`
Extends	`IsoSmartCardSpi`
Since	3.0
Purpose	Multi-channel variant of the smart-card SPI, used by card extensions whose underlying card supports several active logical channels. Deriving from the ISO facet, it can only be produced by an `IsoCardSelectionExtensionSpi`.

Get Channel

Signature	`getChannel() → Int`
Since	3.0
Description	Returns the logical channel number on which this smart card has been placed.
Returns	A non-negative `Int`.
Pre-conditions	None.
Errors	None.

5.4.6. Smart Card SPI

Name	`SmartCardSpi`
Kind	Interface (SPI)
Namespace	`card.spi`
Since	1.0
Purpose	Card-extension facet of a successfully selected smart card.

Deactivate

Signature	`deactivate() → Unit`
Since	3.0
Description	Marks this smart card as no longer active. After this operation has returned, the corresponding `SmartCard` exposed to the application via CNA-TR-API MUST report `isActive() == false`. MUST be idempotent: calling it twice MUST have the same observable effect as calling it once.
Pre-conditions	None.
Errors	None.

5.5. Errors

Every error defined below carries two properties: `message` (`String`), which describes the condition, and `cause` (`Any?`), the underlying error that triggered it, `null` when there is none. They are not restated for each error.

The APDU-level errors—`ReaderBrokenCommunication`, `CardBrokenCommunication`, `UnexpectedStatusWord` and `ApduExchangeDurationExceeded`—additionally carry `cardResponse` (`CardResponse?`), the responses collected before the failure occurred, and `isCardResponseComplete` (`Boolean`), which indicates whether every prepared APDU produced a response.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

5.5.1. APDU Exchange Duration Exceeded

Name	`ApduExchangeDurationExceeded`
Kind	Error
Namespace	`card`
Since	3.0
Purpose	Indicates that the effective duration of an APDU exchange exceeded the bound declared on the request via `ApduRequest.apduExchangeMaxDuration`. Higher-level extensions (e.g. Calypso) MAY intercept this error, cancel any ongoing session and propagate the failure to the application as an `InvalidCardResponse` (CNA-TR-API).

5.5.2. Card Broken Communication

Name	`CardBrokenCommunication`
Kind	Error
Namespace	`card`
Since	1.0
Purpose	Indicates that the communication with the card has failed during the execution of a card request, or that the targeted smart card is no longer active. Carries the response data received from the card until the failure occurred.

5.5.3. Parse

Name	`Parse`
Kind	Error
Namespace	`card`
Since	2.0
Purpose	Raised by the `parse` operation of a selection SPI, `BasicCardSelectionExtensionSpi` or `IsoCardSelectionExtensionSpi`, when the parsing of the card selection response has failed. The most likely reason is that the **Select Application** command returned an invalid FCI structure.

5.5.4. Reader Broken Communication

Name	`ReaderBrokenCommunication`
Kind	Error
Namespace	`card`
Since	1.0
Purpose	Indicates that the communication with the reader has failed during the execution of a card request. Carries the response data received from the card until the failure occurred.

5.5.5. Unexpected Status Word

Name	`UnexpectedStatusWord`
Kind	Error
Namespace	`card`
Since	1.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Purpose	Indicates that a command returned a status word that was not part of its `successfulStatusWords` set. Carries the response data received from the card until that status word was received.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement